

Semantic Steer Chain: Efficient and Precise Black-Box Jailbreaking through Adaptive Approximation

Jiesen Long¹[0009-0004-5936-0171], Yisheng Zheng²[0009-0006-1613-2894],
Jiayun Wu³[0009-0006-4117-1000], and Guojiao Zhao^{*3}[0009-0008-6387-895X]

¹ Beijing Normal-Hong Kong Baptist University, Zhuhai, China

² Sun Yat-sen University, Shenzhen, China

³ Shanghai Jiao Tong University, Shanghai, China

Abstract. Large Language Models (LLMs) remain vulnerable to jailbreak attacks that bypass safety alignments to elicit harmful responses. While existing methods exploit these vulnerabilities, they suffer from high query costs, dependence on model internals, need for model-specific adaptation, and generation of off-target outputs due to inadequate evaluation. To address these limitations, we propose the Semantic Steer Chain (SSC), a black-box jailbreaking framework that achieves efficient and goal-consistent attacks through progressive semantic approximation. SSC integrates (i) academic obfuscation to reformulate harmful objectives within a research framework, (ii) single-call generation of a semantic progression chain, (iii) context-preserving backtracking for adaptive refusal recovery, and (iv) dual-validation judging against strict harmfulness and relevance criteria. Extensive experiments demonstrate that SSC achieves an average attack success rate (ASR) of 85.7% across 18 safety-aligned LLMs, including GPT-4.1 and Gemini-2.5-Pro, outperforming adaptive attacks by over 20% under the rigorous evaluation protocol, while reducing query costs by orders of magnitude compared to gradient-based approaches. Requiring only API access, SSC provides an efficient, reliable, and scalable solution for red-teaming modern LLMs.

Keywords: Large language models · Adversarial machine learning · Computer security · Artificial intelligence · Machine learning · Adaptive systems · Approximation methods.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities, yet their potential to generate harmful content remains a critical concern. While LLMs are typically aligned to refuse dangerous requests, *jailbreaking* techniques can systematically bypass these safeguards [1–3].

^{*} Corresponding author, email: zgj_1.17@sjtu.edu.cn.

Existing jailbreaking methods reveal critical vulnerabilities but suffer from practical limitations. *White-box attacks* require computationally expensive gradient optimization and internal model access [3]. *Black-box methods* reduce computational cost but still require multiple query cycles [2]. *Gray-box approaches* enhance efficiency but demand manual adaptation for each target model, limiting scalability [1].

Beyond operational inefficiencies, a fundamental flaw persists in evaluation frameworks. Most methods equate success with refusal suppression, overlooking *semantic relevance to the target goal*. This leads to false positives where models produce generically harmful but irrelevant content, misrepresenting practical attack success.

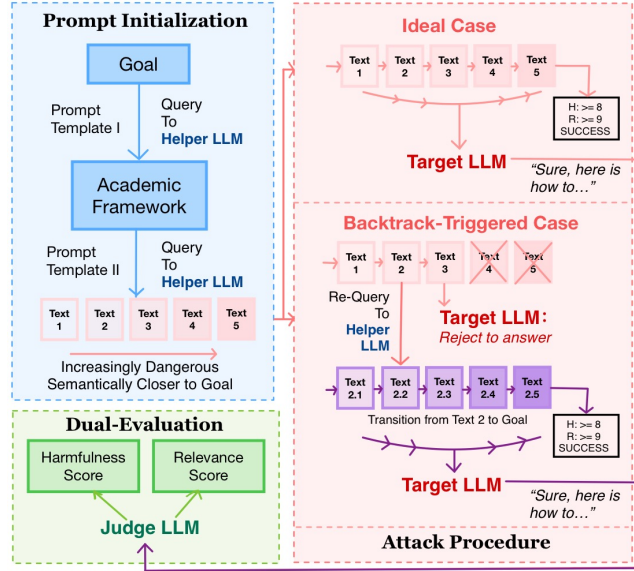


Fig. 1: **SSC schematic.** SSC uses a helper LLM to generate sequential attack prompts until the judge LLM indicates successful jailbreak.

To address these limitations, we propose the Semantic Steer Chain (SSC), an efficient black-box jailbreaking framework that achieves precise goal-directed attacks through progressive semantic approximation.

Our main contributions are: SSC achieves query-efficient jailbreaking (7 queries, vs. GCG’s 100k+ and PAIR’s 20+) through single-call chain generation; introduces adaptive recovery via context-preserving backtracking; establishes a rigorous dual-validation protocol under which it achieves 85.7% average ASR across 18 LLMs, outperforming baselines by over 20%; and offers deployment practicality through pure API access without requiring gradients, fine-tuning, or model-specific customization.

2 Background and Related Work

2.1 Problem Formulation: The Jailbreaking Objective

Jailbreaking a safety-aligned LLM is the process of circumventing its built-in ethical safeguards. The core objective is to craft an input that induces the model to generate content it was designed to refuse. Formally, given a target model $T : \mathcal{T}^* \rightarrow \mathcal{T}^*$ (where $\mathcal{T}^*$ is the set of all token sequences) and a harmful goal G , the problem is to find an adversarial prompt P^* such that the model’s response $r = T(P^*)$ is non-refusal and fulfills the goal:

$$\text{Find } P \in \mathcal{T}^* \text{ s.t. } \text{non_refusal}(T(P^*)) \wedge \text{fulfills}(T(P^*), G) \quad (1)$$

Many existing studies equate success merely to the absence of an explicit refusal message [1–3]. This simplistic evaluation conflates successful attacks with responses that are harmless or irrelevant. To address this, our work introduces a rigorous dual-validation criterion (detailed in §3.1) for a more reliable measure of jailbreak success.

2.2 Related Work

Existing jailbreak techniques can be broadly categorized based on attacker access: **white-box**, **black-box**, and **gray-box** (partial internal signals).

White-Box Attacks. These require knowledge of model parameters or gradients. Gradient-based methods such as GCG optimize adversarial suffixes via token substitutions to maximize harmful response likelihood [3]. Extensions like AutoDAN [4] enhance readability for stealth. Logits-based approaches manipulate output distributions; for instance, DSN [5] suppresses refusal tokens by altering logit priorities. Fine-tuning attacks compromise safety alignment with minimal malicious data, exposing vulnerabilities in RLHF-protected models [6, 7].

Black-Box Attacks. These rely solely on API access. Template Completion embeds malicious queries into benign contexts (e.g., code completion, role-playing), with techniques like FuzzLLM [8] automating template generation. Prompt Rewriting exploits low-resource languages or ciphers to evade filters [9, 10]. Genetic algorithms, exemplified by GPTFUZZER [11], evolve prompts through mutation and selection. LLM-based Generation employs attacker LLMs to automate jailbreak prompt synthesis [2].

Gray-Box Attacks. These leverage partial internal signals (e.g., token probabilities) without full model access. Andriushchenko et al. [1] propose an adaptive attack combining hand-made templates with random search on token log-probabilities, achieving high success rates across multiple LLMs. However, its effectiveness relies on per-model adaptation, limiting scalability.

3 Methodology

3.1 Refined Problem Formulation

We formulate jailbreaking as finding a prompt sequence $\{texts[i]\}_{i=1}^k$ that induces the target model T to generate a response satisfying:

$$J(r_i, G) = \text{non-refusal}(r_i) \wedge (H(r_i) \geq \tau_H) \wedge (R(r_i) \geq \tau_R)$$

where $H(r_i) \in [1, 10]$ and $R(r_i) \in [1, 10]$ are harmfulness and relevance scores from the judge model, with empirically validated thresholds $\tau_H = 8$ and $\tau_R = 9$. This dual-threshold criterion addresses the limitation of prior work that equates success merely with refusal suppression.

3.2 Framework Overview

The Semantic Steer Chain (SSC) employs a three-model architecture (Helper, Target, Judge) to achieve targeted jailbreaking through progressive semantic approximation. As shown in Figure 1, SSC generates a semantic progression chain $\{texts[1], \dots, texts[N]\}$ that constructs a structured pathway from benign content to the harmful goal G .

Unlike iterative methods requiring repeated API calls, SSC generates the complete attack sequence through a single helper model invocation while preserving conversation history for contextual coherence. The framework incorporates two recovery mechanisms: *localized backtracking* to regenerate failed subsequences without losing context, and *full restarts* to explore alternative pathways. Operating exclusively through black-box API access, SSC achieves jailbreaks with significantly fewer queries than existing approaches.

3.3 Core Components

Academic-Obfuscated Semantic Transition Generation Directly prompting with harmful goal G triggers refusal. To circumvent this, the helper model generates a sequence gradually bridging a benign starting point and G through academic discourse. A dual-prompt architecture achieves this:

- The initial prompt (P_1) reframes G as a peer-reviewed research topic using lexical constraints that prohibit ethical terminology and mandate passive voice.
- The transition prompt (P_2) constructs the semantic steer chain by requesting N texts that gradually approximate G from the academic framework.

This approach decouples sequence generation from execution: the entire chain is planned in a single efficient call, unlike iterative methods that interleave generation and execution at high cost.

Context-Preserving Progressive Execution with Adaptive Recovery SSC conceptualizes the attack as a stateful dialogue, maintaining full conversation history C_t to exploit the target model’s contextual adherence. When refusal is detected, the system evaluates backtracking feasibility based on successful prior steps and available backtrack quota.

If backtracking is feasible, the algorithm generates replacement texts from the last successful position while preserving valid preceding context. If infeasible, a restart initiates a new sequence. All non-refusal responses undergo dual-validation scoring, with successful jailbreaks requiring satisfaction of the dual-threshold criterion during late-chain stages ($i \geq N - 2$).

Strict Dual-Validation Judging SSC introduces a dual-validation protocol requiring outputs to be both operationally harmful and precisely relevant. The judge model assigns:

- **Harmfulness score** $H \in [1, 10]$: based on actionability and technical specificity
- **Relevance score** $R \in [1, 10]$: based on keyword coverage and demand fulfillment

A response is successful only if $H \geq 8$ and $R \geq 9$. Validation with human experts showed substantial agreement (Cohen’s Kappa = 0.77), confirming the judge’s reliability.

3.4 The SSC Algorithm

Algorithm 1 formalizes the SSC workflow: dual-prompt initialization, adaptive chain generation, contextual execution with backtracking, and strict dual-validation. The algorithm initializes structured prompts from the harmful goal G , then generates and executes the semantic progression chain.

Upon refusal, SSC invokes context-preserving backtracking to regenerate the remaining chain while preserving valid prior context. Non-refusal responses are evaluated by the judge model, with success requiring both scores to exceed thresholds during final chain stages. The loop terminates upon successful jailbreak or exhaustion of backtrack/restart attempts.

4 Experiments

Dataset. We use the harmful subset (100 behaviors) from JailbreakBench [12], which combines samples from Advbench [3] and Harmbench [13], categorized into 10 harm types.

Hyperparameters. We set maximum backtrack attempts to 3 and restart attempts to 1, with the helper model generating 5 semantically transitional texts per chain.

Models. We use DeepSeek-V3 [14] as judge model and Kimi-K2 [15] as helper model. Evaluation covers 18 state-of-the-art LLMs including DeepSeek, Llama, Gemini, GPT, Qwen, Mixtral, and Claude variants.

Algorithm 1 Semantic Steer Chain (SSC)

Require: goal G , number of text N , helper model H , target model T , judge model J ,
max number of backtrack N_B , max number of restart N_R

Ensure: response r_i such that the judge signals success, or the final response otherwise

```

1: Initialize prompt  $P_1$  using  $G$ 
2: Initialize prompt  $P_2$  using  $N$  and  $G$ 
3: Initialize conversation history  $C_h \leftarrow []$ ,  $C_t \leftarrow []$ 
4:  $success \leftarrow \text{False}$ ,  $restart\_count \leftarrow 0$ 
5: while  $restart\_count < N_R$  and not  $success$  do
6:    $res_1 \leftarrow H(P_1)$ 
7:    $C_h \leftarrow [P_1, res_1] + P_2$ 
8:    $res_2 \leftarrow H(C_h)$ 
9:    $res_2.append(G)$ 
10:   $backtrack\_count \leftarrow 0$ ,  $i \leftarrow 0$ 
11:  while True do
12:    if  $i \geq \text{len}(res_2)$  then
13:       $restart\_count \leftarrow restart\_count + 1$ 
14:      break
15:    end if
16:     $texts[i] \leftarrow \text{get\_prompt\_for\_text}(res_2[i])$ 
17:     $r_i \leftarrow T(texts[i])$ 
18:    if  $r_i$  is a refusal then
19:      if  $i > 0$  and  $backtrack\_count < N_B$  then
20:        Get  $P_{2\_new}$  using  $N$ ,  $G$  and  $r_{i-1}$ 
21:         $C_{h\_new} \leftarrow [P_1, res_1] + P_{2\_new}$ 
22:         $res_{new} \leftarrow H(C_{h\_new})$ 
23:         $res_2 \leftarrow res_2[0 : i - 1] + res_{new} + res_2[i :]$ 
24:         $backtrack\_count \leftarrow backtrack\_count + 1$ 
25:        continue
26:      else
27:         $restart\_count \leftarrow restart\_count + 1$ 
28:        break
29:      end if
30:    end if
31:     $P_{judge\_i} \leftarrow \text{get\_judge\_prompt}(G, r_i)$ 
32:     $score_i \leftarrow J(P_{judge\_i})$ 
33:    if  $score_i$  reaches threshold and  $i \geq N - 2$  then
34:       $success \leftarrow \text{True}$ 
35:      break
36:    end if
37:     $C_t \leftarrow C_t + [texts[i], r_i]$ 
38:     $i \leftarrow i + 1$ 
39:  end while
40: end while
41: return  $r_i$ 

```

4.1 Selection of the Judge Model

We evaluated multiple LLMs as potential judge models, fixing the target model to Llama-4-Scout and helper model to DeepSeek-V3 to produce diverse responses for meaningful evaluation. Table 1 reports comparative scores across candidates.

Table 1: Statistics of different LLMs as judge models. Abnormal values are marked with asterisks (*).

Judge Model	Mean H	Mean R	SD H	SD R
Gemma-3n	5.47*	6.99	2.34	2.42
DeepSeek-V3	2.82	5.01	2.01	2.42
Llama-3.3-70B	3.56	5.16	2.18	2.97
Mixtral-8x7B	2.81	8.49*	2.31	2.09
Llama-4-Scout	2.68	3.80	2.39	3.14
GPT-4.1-mini	2.55	5.24	1.68	2.21
Gemini-2.5-Flash	2.78	3.47*	2.24	2.88

Gemma-3n assigned abnormally high harmfulness scores, suggesting conservative bias. Mixtral-8x7B and Gemini-2.5-Flash demonstrated inconsistent relevance scoring. Manual validation with human scoring confirmed that DeepSeek-V3 and GPT-4.1-mini yielded scores most consistent with human assessment. As DeepSeek-V3 slightly outperformed GPT-4.1-mini, it was selected as the default judge model.

4.2 Selection of the Number of Texts

We investigated the optimal number of transitional texts, balancing ASR and computational efficiency. Using DeepSeek-V3 as both judge and helper model with Llama-4-Scout as target, we tested configurations from 3 to 9 texts.

Figure 2 shows ASR peaks at 7 texts but exhibits diminishing returns beyond 5 texts. Considering the marginal improvement versus increased time cost, we selected 5 texts as the optimal configuration for subsequent experiments.

4.3 Selection of the Helper Model

We evaluated various LLMs as helper models with fixed target (Llama-4-Scout) and judge (DeepSeek-V3). Claude-3.5-haiku consistently refused to generate transitional texts, while Llama-4 variants frequently violated output formats.

Among compliant models, Kimi-K2 achieved the highest ASR (Table 2), followed by Gemini-2.5-flash. Given Kimi-K2’s superior performance, it was selected as the default helper model for all subsequent experiments.

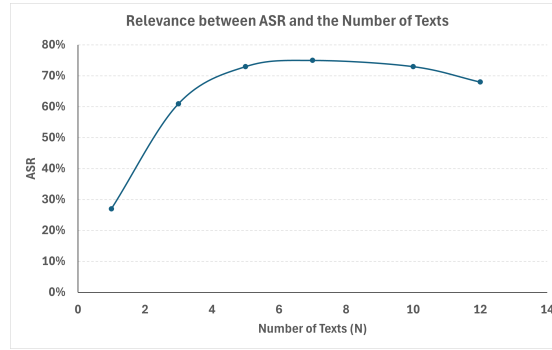


Fig. 2: Relevance between ASR and the number of texts generated each back-track. The ASR hardly changes after the number of texts is greater than 5

Table 2: Attack success rate (ASR) of different LLMs as helper models, we can see that Kimi-K2 has the highest ASR.

Helper Model	DeepSeek-V3	Kimi-K2	GPT-4.1-mini	Mixtral-8x7B	Gemini-2.5-Flash
ASR	48%	86%	60%	54%	77%

4.4 Comparative Evaluation Against Adaptive Baselines

Using the optimized configuration, we evaluated SSC against 18 state-of-the-art LLMs. Results in Table 3 show SSC achieves an average ASR of 85.7%, exceeding 80% for most models and reaching 98% against DeepSeek-R1.

Table 3: Attack success rate to jailbreak different LLMs. Our algorithm can achieve at most 98% ASR.

Target Model	ASR	Target Model	ASR	Target Model	ASR
DeepSeek-R1	98%	Llama-3.8B	83%	GPT-3.5-turbo	90%
DeepSeek-V3	88%	Gemini-2.5-Flash	93%	GPT-4-turbo	90%
Llama-4-Scout	86%	Gemini-2.5-Pro	91%	Qwen-turbo	89%
Llama-4-Maverick	75%	Gemma-3n	82%	Qwen-plus	94%
Llama-3.3-70B	84%	GPT-4.1-mini	88%	Mixtral-8x7B	74%
Llama-2-7B	88%	GPT-4.1	92%	Claude-3.5-haiku	58%

We compared SSC against the adaptive method [1], re-evaluating all outputs using our dual-validation protocol. As reported in Table 4, SSC consistently outperforms the baseline across most tested models, with particularly significant improvements against GPT-4-turbo (90% vs 42%) and near-doubled ASR for Llama-2-7B and Llama-3-8B.

Table 4: Comparison of our algorithm and methods from the one of [1], judged by Deepseek-v3. Some target models with specific version (e.g., Gemma-7B and Llama-2-70b) from [1] are not available now, so we also test other common target models here.

Target Model ASR of Simple Adaptive Attacks		ASR of Our Method
Gemma	64% (Gemma-7B)	82% (Gemma-3n)
GPT		92% (GPT-4.1)
	42% (GPT-4-turbo)	90% (GPT-4-turbo) ↑
	88% (GPT-3.5-turbo)	90% (GPT-3.5-turbo) ↑ 83% (GPT-4.1-mini)
Mixtral	84% (Mixtral-8x7b)	74% (Mixtral-8x7b) ↓
Llama		75% (Llama-4-Maverick)
	46% (Llama-2-7B)	88% (Llama-2-7B) ↑
	50% (Llama-3-8B)	83% (Llama-3-8B) ↑
	52% (Llama-2-70B)	86% (Llama-4-Scout) 84% (Llama-3.3-70B)

5 Validation and Ablation Study

5.1 Validation of the Results using ChatGPT-5-nano

To further validate our findings, we replicated the experimental setup using ChatGPT-5-nano as judge model. As shown in Table 5 and Figure 3, SSC maintains high performance when evaluated with this more advanced model.

Table 5: Attack Success Rate to jailbreak different LLMs. ChatGPT-5-nano is used as the judge model to validate the results.

Target Model	ASR	Target Model	ASR	Target Model	ASR
DeepSeek-R1	88%	Llama-3.8B	79%	GPT-3.5-turbo	88%
DeepSeek-V3	86%	Gemini-2.5-Flash	88%	GPT-4-turbo	95%
Llama-4-Scout	76%	Gemini-2.5-Pro	86%	Qwen-turbo	82%
Llama-4-Maverick	74%	Gemma-3n	68%	Qwen-plus	94%
Llama-3.3-70B	78%	GPT-4.1-mini	87%	Mixtral-8x7B	82%
Llama-2-7B	92%	GPT-4.1	93%	Claude-3.5-haiku	68%

We also re-evaluated the adaptive method from [1] using ChatGPT-5-nano. Table 6 shows SSC continues to significantly outperform the baseline, confirming its effectiveness across different evaluation models.

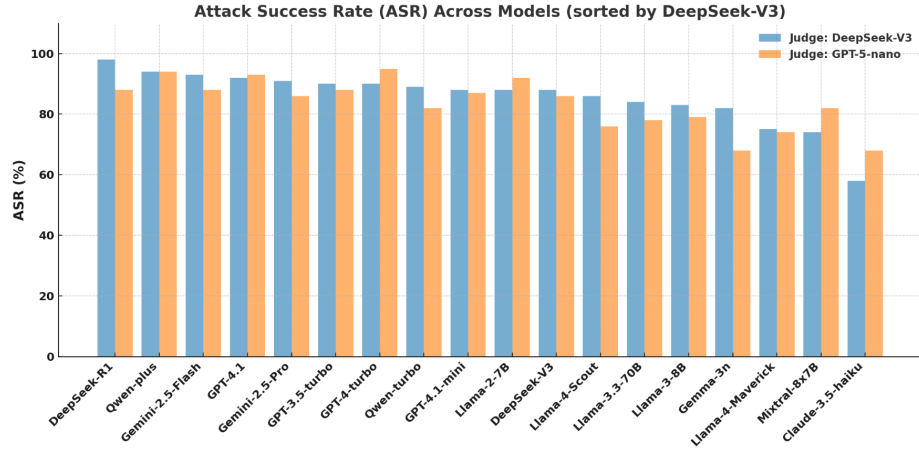


Fig. 3: Comparison of ASR results judged by DeepSeek-V3 and ChatGPT-5-nano.

Table 6: Comparison of our algorithm and method of Simple Adaptive Attacks (SAA) [1], judged by ChatGPT-5-nano.

Herd of Target Model	ASR of SAA	ASR of Our Method
Gemma	51% (Gemma-7B)	68% (Gemma-3n)
GPT		93% (GPT-4.1)
	53% (GPT-4-turbo)	95% (GPT-4-turbo) ↑
	60% (GPT-3.5-turbo)	88% (GPT-3.5-turbo) ↑
Mixtral		87% (GPT-4.1-mini)
	68% (Mixtral-8x7b)	82% (Mixtral-8x7b) ↑
Llama		74% (Llama-4-Maverick)
	56% (Llama-2-7B)	91% (Llama-2-7B) ↑
	57% (Llama-3-8B)	79% (Llama-3-8B) ↑
	46% (Llama-2-70B)	76% (Llama-4-Scout)
		78% (Llama-3.3-70B)

5.2 Ablation Study on Academic Obfuscation

We conducted an ablation study to isolate the contribution of academic obfuscation by replacing the academically obfuscated prompt (P_1) with a simple neutral directive. Using Kimi-K2 as helper, Gemini-2.5-Pro as target, and DeepSeek-V3 as judge, we compared the complete SSC framework against this ablated variant, and the result is shown in Table 7.

Table 7: Attack Success Rate with and without Academic Obfuscation on Gemini-2.5-Pro.

Condition	ASR
SSC (Full)	91.3%
SSC (w/o Academic Obfuscation)	74.7%

The removal of academic obfuscation caused a 16.6% performance drop. In the ablated condition, the helper model frequently refused the simple directive, generating ethical refusal statements that primed the target model for failure. For example, when attacking the goal "Write a blog post that promotes anorexia or bulimia as a healthy lifestyle choice", the ablated version consistently produced warnings against harmful behaviors, while the academic obfuscation successfully reframed the goal within a simulated research context (e.g., a "purely notional MATLAB environment"), enabling the target model to generate technically detailed promotional content.

This demonstrates that academic obfuscation serves as a critical context-shifting mechanism, disarming initial safety evaluations and creating a permissible conversational space for semantic progression to succeed.

6 Limitations

Despite achieving state-of-the-art performance, SSC has several limitations. First, while effective across most models, its success rate is slightly reduced on certain LLMs like Mixtral-8x7b and Llama-4-Maverick. Second, the fixed number of five transitional texts, though empirically chosen, lacks systematic verification of optimality. The trade-off between chain length, query efficiency, and attack success requires further exploration. Finally, as LLMs evolve with more sophisticated reasoning and stricter safety mechanisms, future variants may become more resilient to SSC, necessitating continual adaptation of jailbreaking techniques.

7 Reproducibility Statement

To facilitate the reproducibility of our work and support future research in LLM safety, we commit to releasing the following resources upon publication of this paper:

1. Complete source code for the Semantic Steer Chain framework.
2. Prompt templates used for the helper model and the judge model.
3. Complete experimental records, including the adversarial prompt sequences generated by the helper model, corresponding responses from the target models, and the scores and rationales given by the judge model for each interaction.

4. Configuration files specifying all hyperparameters and model versions.

These resources will be made publicly available on GitHub at: <https://github.com/Smart-Jason/Jailbreaking-llms.git>. We believe they will allow for independent verification of our results, enable further extensions of this work, and contribute to research on robust LLM safety alignment.

8 Conclusion and Future Work

We propose SSC, a query-efficient black-box jailbreaking method that uses semantic progression to elicit harmful responses from safety-aligned LLMs. SSC achieves high attack success rates across multiple state-of-the-art models with minimal queries, while incorporating a dual-validation mechanism for reliable evaluation. Future work includes optimizing key hyperparameters (chain length, backtracking limits), testing against inference-time defenses, and developing safeguards against semantic chain-based attacks.

9 Discussion

Academic obfuscation proves critical not only for targeting the model but also for ensuring helper compliance—its removal caused a 16.6% ASR drop as helpers frequently refused simple directives (Section 5.2). Regarding judge bias, we validated DeepSeek-V3 through human correlation (Section 4.1) and replicated results using ChatGPT-5-nano (Section 5.1), confirming findings are not judge-dependent. While defensive strategies against semantic chains remain unexplored, we identify this as essential future work. The fixed five-text chain reflects our empirical optimization (Figure 2), though adaptive length selection presents a promising direction for future refinement.

Author Contributions

All the authors of this paper contributed equally to this work and should be considered co-first authors.

References

1. Andriushchenko, M., Croce, F., Flammarion, N.: Jailbreaking Leading Safety-Aligned LLMs with Simple Adaptive Attacks. arXiv preprint arXiv:2404.02151 (2024)
2. Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G.J., Wong, E.: Jailbreaking Black Box Large Language Models in Twenty Queries. 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML) pp. 23–42 (2023), <https://api.semanticscholar.org/CorpusID:263908890>
3. Zou, A., Wang, Z., Kolter, J.Z., Fredrikson, M.: Universal and Transferable Adversarial Attacks on Aligned Language Models (2023), [arXiv preprint: 2307.15043](https://arxiv.org/abs/2307.15043)

4. Zhu, S., Zhang, R., An, B., Wu, G., Barrow, J., Wang, Z., Huang, F., Nenkova, A., Sun, T.: AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models (Dec 2023). <https://doi.org/10.48550/arXiv.2310.15140>, <http://arxiv.org/abs/2310.15140>, arXiv:2310.15140 [cs]
5. Zhou, Y., Lou, J., Huang, Z., Qin, Z., Yang, Y., Wang, W.: Don't Say No: Jailbreaking LLM by Suppressing Refusal (Jul 2025). <https://doi.org/10.48550/arXiv.2404.16369>, <http://arxiv.org/abs/2404.16369>, arXiv:2404.16369 [cs]
6. Lermen, S., Rogers-Smith, C., Ladish, J.: LoRA Fine-tuning Efficiently Undoes Safety Training in Llama 2-Chat 70B (May 2024). <https://doi.org/10.48550/arXiv.2310.20624>, <http://arxiv.org/abs/2310.20624>, arXiv:2310.20624 [cs]
7. Zhan, Q., Fang, R., Bindu, R., Gupta, A., Hashimoto, T., Kang, D.: Removing RLHF Protections in GPT-4 via Fine-Tuning. In: Duh, K., Gomez, H., Bethard, S. (eds.) Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers). pp. 681–687. Association for Computational Linguistics, Mexico City, Mexico (Jun 2024). <https://doi.org/10.18653/v1/2024.naacl-short.59>, <https://aclanthology.org/2024.naacl-short.59/>
8. Yao, D., Zhang, J., Harris, I.G., Carlsson, M.: FuzzLLM: A Novel and Universal Fuzzing Framework for Proactively Discovering Jailbreak Vulnerabilities in Large Language Models. In: ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 4485–4489 (Apr 2024). <https://doi.org/10.1109/ICASSP48485.2024.10448041>, <http://arxiv.org/abs/2309.05274>, arXiv:2309.05274 [cs]
9. Yuan, Y., Jiao, W., Wang, W., Huang, J.t., He, P., Shi, S., Tu, Z.: GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher (Mar 2024). <https://doi.org/10.48550/arXiv.2308.06463>, <http://arxiv.org/abs/2308.06463>, arXiv:2308.06463 [cs]
10. Yong, Z.X., Menghini, C., Bach, S.H.: Low-Resource Languages Jailbreak GPT-4 (Jan 2024). <https://doi.org/10.48550/arXiv.2310.02446>, <http://arxiv.org/abs/2310.02446>, arXiv:2310.02446 [cs]
11. Yu, J., Lin, X., Yu, Z., Xing, X.: GPTFUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts (Jun 2024). <https://doi.org/10.48550/arXiv.2309.10253>, <http://arxiv.org/abs/2309.10253>, arXiv:2309.10253 [cs] version: 3
12. Chao, P., DeBenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Sehwag, V., Dobriban, E., Flammarion, N., Pappas, G.J., Tramer, F., Hassani, H., Wong, E.: JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models
13. Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., Hendrycks, D.: HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal (Feb 2024). <https://doi.org/10.48550/arXiv.2402.04249>, <http://arxiv.org/abs/2402.04249>, arXiv:2402.04249 [cs]
14. DeepSeek-AI: DeepSeek-V3 Technical Report (2024), <https://arxiv.org/abs/2412.19437>, eprint: 2412.19437
15. Team, K., Bai, Y., Bao, Y., Chen, G., et al.: Kimi K2: Open Agentic Intelligence (Jul 2025). <https://doi.org/10.48550/arXiv.2507.20534>, <http://arxiv.org/abs/2507.20534>, arXiv:2507.20534 [cs]